

CEFAS L3 GENIE INFO

1



spring boot

HAMADOU MBALLO

- INGENIEUR INFORMATICIEN
- FORMATEUR INFORMATIQUE
- DEV FULL STACK

INTRODUCTION

2

- Développer en Java n'est pas toujours chose facile! Tous les développeurs Java se sont déjà sentis découragés, face à des projets de développement complexes techniquement et fonctionnellement, à une masse de code à optimiser, à des librairies incompréhensibles...
- Heureusement, il y a une solution : **Spring Framework** et **Spring Boot** ! Spring Framework et Spring Boot offrent un environnement de développement **solide et efficace**, qui va vous simplifier la vie. La gestion des **dépendances**, la **configuration**, la gestion des **propriétés** et le **déploiement** seront bientôt des jeux d'enfants !
- Avec ce cours, vous découvrirez tous les **avantages de Spring Boot** et saurez précisément **quand et pourquoi** l'utiliser. Vous saurez appréhender n'importe quel projet avec Spring Boot de A à Z, en passant par **quatre étapes clés** : créer votre projet, le structurer et le configurer, écrire votre code, et tester puis déployer.

POURQUOI SPRING BOOT ?

3

- ▶ Il permet de simplifier le démarrage et le développement des nouvelles applications en réduisant la **complexité de configuration**.
- ▶ Dans Spring Boot, tout est configuré automatiquement ; aucune configuration manuelle n'est nécessaire.

APP SPRING BOOT vs APP JEE SIMPLE (sans SPB)

```
1  <?xml version="1.0" encoding="UTF-8"?>
2
3  <persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
4      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5      xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
6          http://xmlns.jcp.org/xml/ns/persistence/persistence_2_1.xsd"
7      version="2.1">
8
9      <!-- Definition of the database - used in JpaConfig class -->
10
11      <persistence-unit name="OpenClassroomsDB">
12          <properties>
13              <property name="javax.persistence.jdbc.url" value="jdbc:mysql://localhost:3306/springjpa?serverTimezone=UTC" />
14              <property name="javax.persistence.jdbc.user" value="root" />
15              <property name="javax.persistence.jdbc.password" value="root" />
16              <property name="javax.persistence.jdbc.driver" value="com.mysql.jdbc.Driver" />
17              <property name="hibernate.show_sql" value="true" />
18              <property name="hibernate.format_sql" value="true" />
19              <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/>
20          </properties>
21      </persistence-unit>
22  </persistence>
```



```
1  spring.datasource.url=jdbc:mysql://localhost:3306/springjpa?serverTimezone=UTC
2  spring.datasource.username=root
3  spring.datasource.password=root
4  spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
```

NOTIONS A COMPRENDRE

❖ Inversion de Contrôle / Inversion of Control (IoC)

- L'Inversion de Contrôle est un principe de conception selon lequel le contrôle du flux d'exécution d'un programme est inversé par rapport à la programmation traditionnelle.
Au lieu que le développeur contrôle l'ordre des appels de méthodes, c'est le framework (comme Spring) qui prend en charge ce contrôle.
- Cela permet de découpler les classes et de rendre les composants de l'application plus testables, plus réutilisables et mieux structurés.

❖ Injection de dépendances

- L'Injection de Dépendances est un design pattern (model de conception) qui implémente l'Inversion de Contrôle pour la gestion des dépendances entre les objets. Au lieu que les objets créent eux-mêmes leurs dépendances, celles-ci leur sont fournies (ou "injectées") par un système externe (comme le conteneur IoC de Spring).
- Dans le contexte de Spring, cela signifie que le Framework crée et gère les objets (appelés beans dans Spring) et leurs dépendances plutôt que de laisser les développeurs instancier manuellement chaque objet.

❖ Annotations Spring Boot

8

Pour manipuler une classe par l'IOC Container, **il est nécessaire de l'indiquer explicitement à Spring**. Deux solutions pour cela:

1- **Fichiers XML**: au sein desquels on décrit les classes que Spring doit gérer

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN 2.0//EN"
3   "http://www.springframework.org/dtd/spring-beans-2.0.dtd">
4
5 <beans>
6   <bean id="myBean" class="com.cefes.BeanImpl"/>
7 </beans>
```

2- **Utilisation des annotations**: Une annotation @[nom de l'annotation] peut être ajoutée à une classe, une méthode, un attribut. **L'annotation influe sur le comportement du programme** car elle fournit des métadonnées lors de la compilation ; ces mêmes métadonnées seront utilisées lors de l'exécution.

Exemple:

@Component, @Autowired,
@Qualifier, @Entity...

```
1 package com.cefes.HelloWorld;
2
3
4 import org.springframework.beans.factory.annotation.Qualifier;
5 import org.springframework.stereotype.Component;
6
7 @Component
8 @Qualifier("myBean")
9 public class BeanImpl {
10
11 }
12
```

ETAPES D'UNE APPLICATION SPRING BOOT

?

1- CREER VOTRE PROJET (1/3)

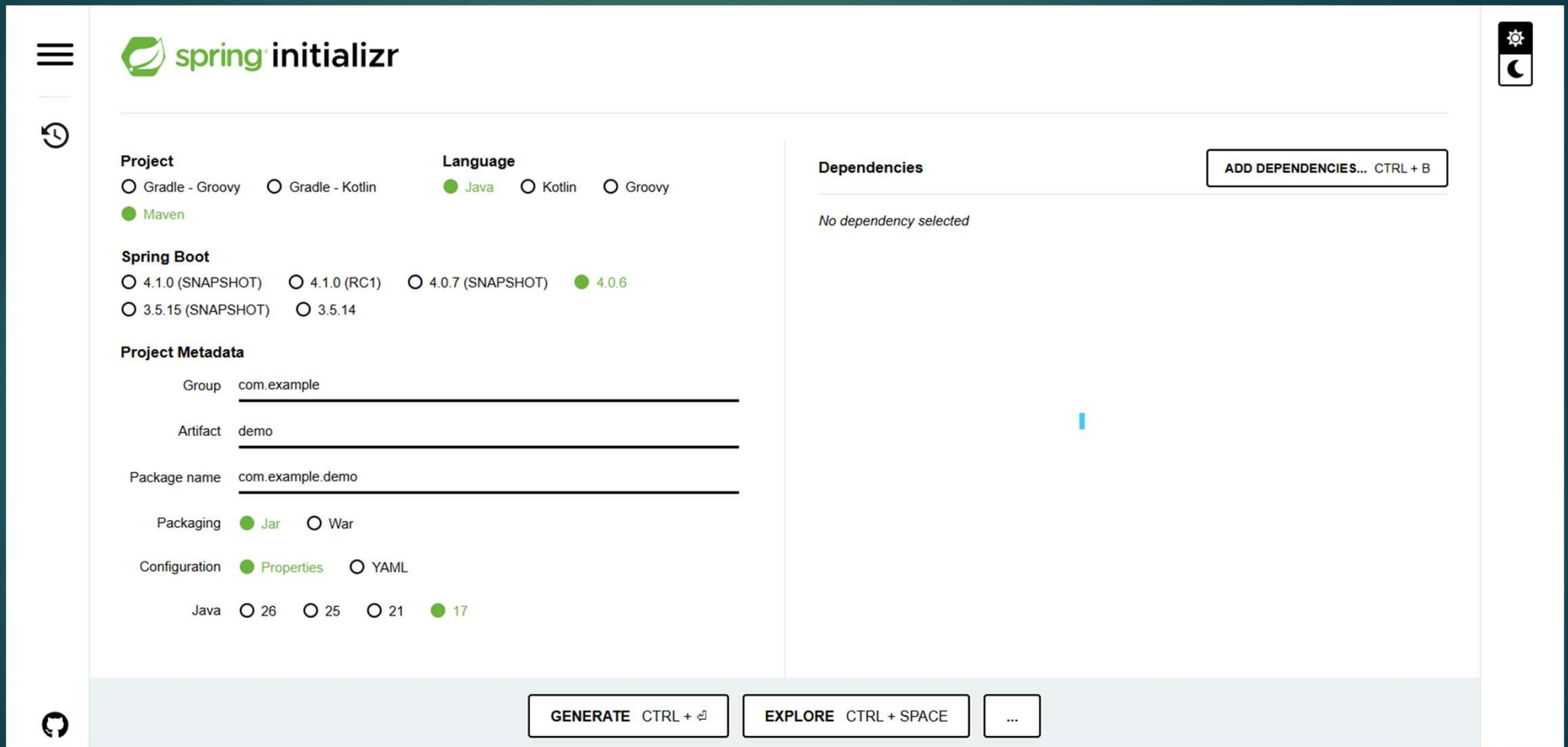
Il existe deux façons de créer un projet Spring Boot :
avec **Spring Initializr** et avec **Spring Tool Suite (STS)**.

➤ <https://start.spring.io/>

➤ <https://spring.io/tools>

1- CREER VOTRE PROJET (Spring Initializr) (2/3)

11



The image shows the Spring Initializr web interface. The header includes the Spring logo and 'spring initializr' text. On the left, there are navigation icons: a hamburger menu, a clock, and a GitHub icon. On the right, there are settings icons: a gear and a moon. The main content area is divided into three sections: 'Project', 'Spring Boot', and 'Project Metadata'. The 'Project' section has radio buttons for 'Gradle - Groovy', 'Gradle - Kotlin', 'Maven' (selected), 'Java' (selected), 'Kotlin', and 'Groovy'. The 'Spring Boot' section has radio buttons for '4.1.0 (SNAPSHOT)', '4.1.0 (RC1)', '4.0.7 (SNAPSHOT)', '4.0.6' (selected), '3.5.15 (SNAPSHOT)', and '3.5.14'. The 'Project Metadata' section has input fields for 'Group' (com.example), 'Artifact' (demo), and 'Package name' (com.example.demo). There are also radio buttons for 'Packaging' ('Jar' selected, 'War' unselected) and 'Configuration' ('Properties' selected, 'YAML' unselected). At the bottom, there are three buttons: 'GENERATE CTRL + G', 'EXPLORE CTRL + SPACE', and an ellipsis button. A 'Dependencies' section on the right shows 'No dependency selected' and an 'ADD DEPENDENCIES... CTRL + B' button.

Project

☐ Gradle - Groovy ☐ Gradle - Kotlin ☒ Maven ☒ Java ☐ Kotlin ☐ Groovy

Spring Boot

☐ 4.1.0 (SNAPSHOT) ☐ 4.1.0 (RC1) ☐ 4.0.7 (SNAPSHOT) ☒ 4.0.6 ☐ 3.5.15 (SNAPSHOT) ☐ 3.5.14

Project Metadata

Group

Artifact

Package name

Packaging ☒ Jar ☐ War

Configuration ☒ Properties ☐ YAML

Java ☐ 26 ☐ 25 ☐ 21 ☒ 17

Dependencies

No dependency selected

ADD DEPENDENCIES... CTRL + B

GENERATE CTRL + G EXPLORE CTRL + SPACE ...

1- CREER VOTRE PROJET (Spring Tool Suite (STS) (3/3))

Spring Tools 4 for Eclipse

The all-new Spring Tool Suite 4. Free.
Open source.

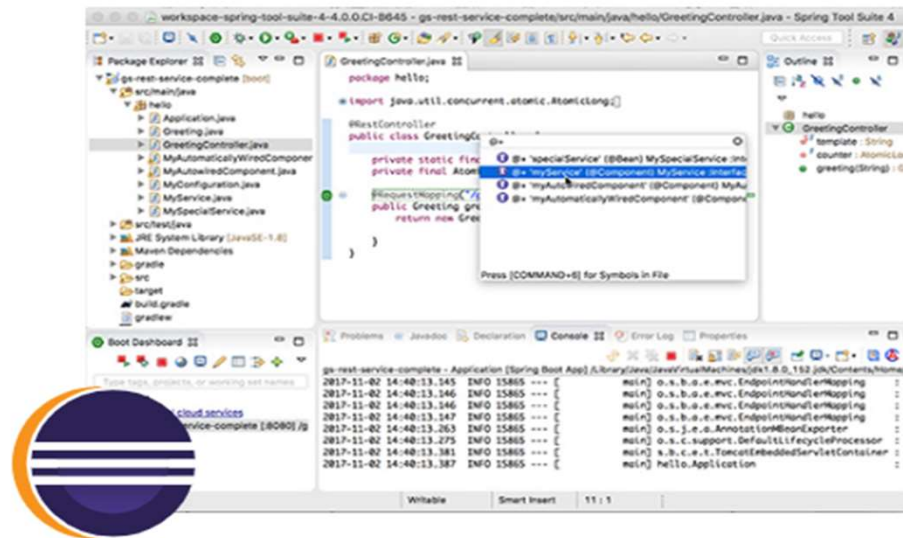
4.19.0 - LINUX X86_64

4.19.0 - LINUX ARM_64

4.19.0 - MACOS X86_64

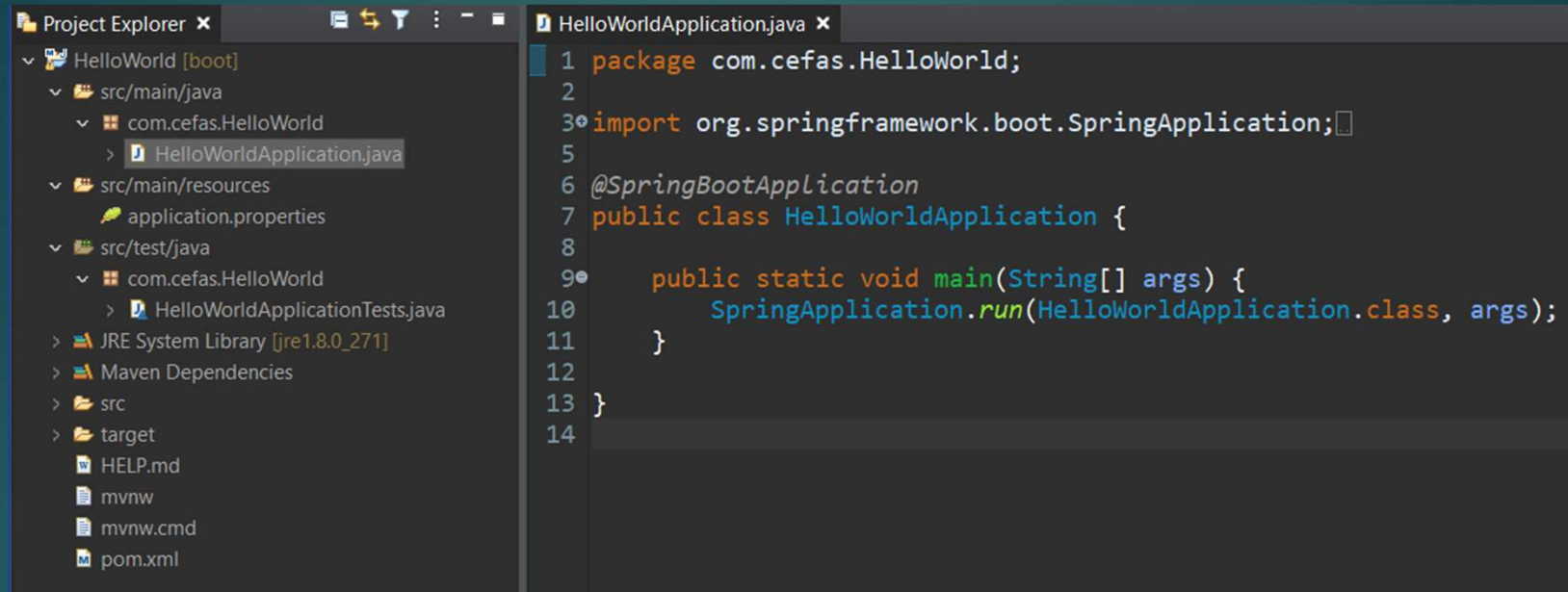
4.19.0 - MACOS ARM_64

4.19.0 - WINDOWS X86_64



❖ Observer la structure minimale du projet

13



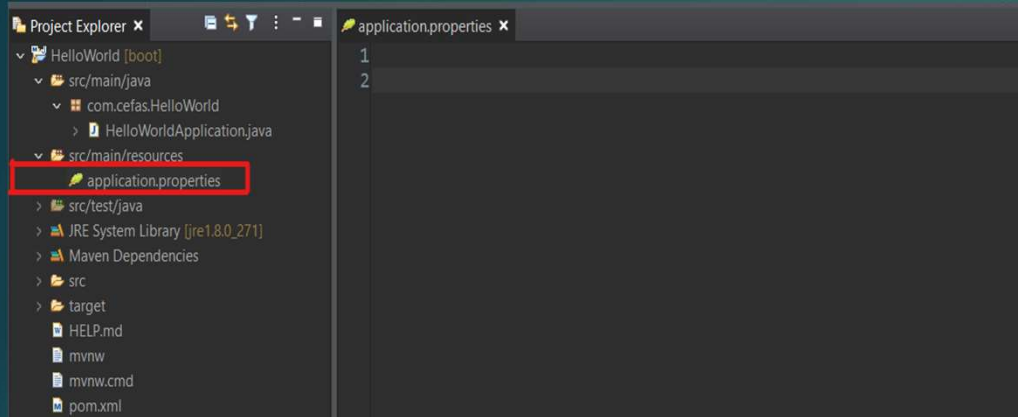
Points à noter :

- Nous retrouvons les fichiers liés à Maven (pom.xml, mvnw et mvnw.cmd).
- Nos sources sont organisées suivant le schéma standard :
 - src/main/java** : contient les packages et les classes Java ;
 - src/main/resources** : contient les fichiers ressources, tels que les fichiers de propriétés ou des templates (HTML, par exemple) ;
 - src/test/java** : contient les classes Java de test.

Comme tout projet Java/Maven, on retrouve également la JRE et les Maven Dependencies

❖ Etape suivante ?

14



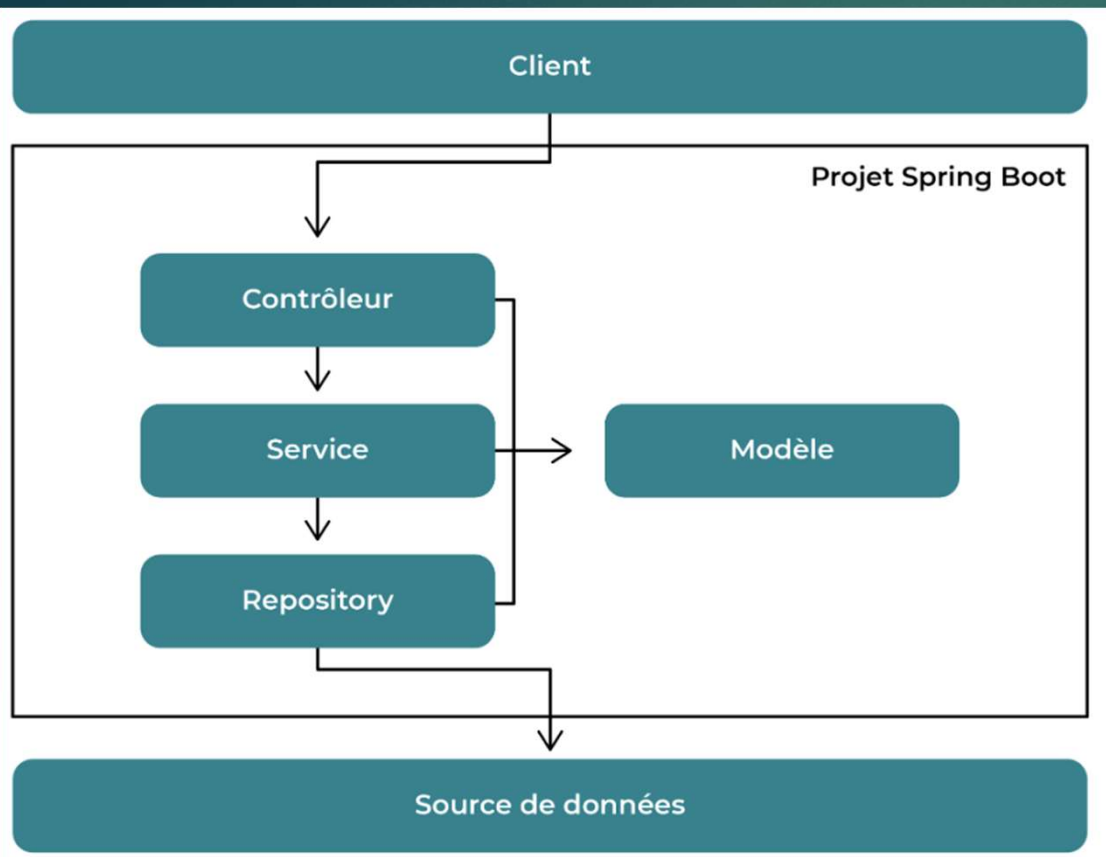
- Un seul package dans **src/main/java**.
On va **créer des sous-packages** pour nos différentes classes à venir.
- Le fichier **applications.properties** est vide: il faut le remplir

Résumé:

- ❖ La structure minimale n'est pas suffisante, il faut l'enrichir.
- ❖ Les **annotations sont un pilier** au sein de Spring Boot pour tirer profit de l'IoC container.
- ❖ L'annotation **@SpringBootApplication**, son objectif est de déclencher toute la mécanique interne de Spring.

2- STRUCTURER ET CONFIGURER LE PROJET(1/2)

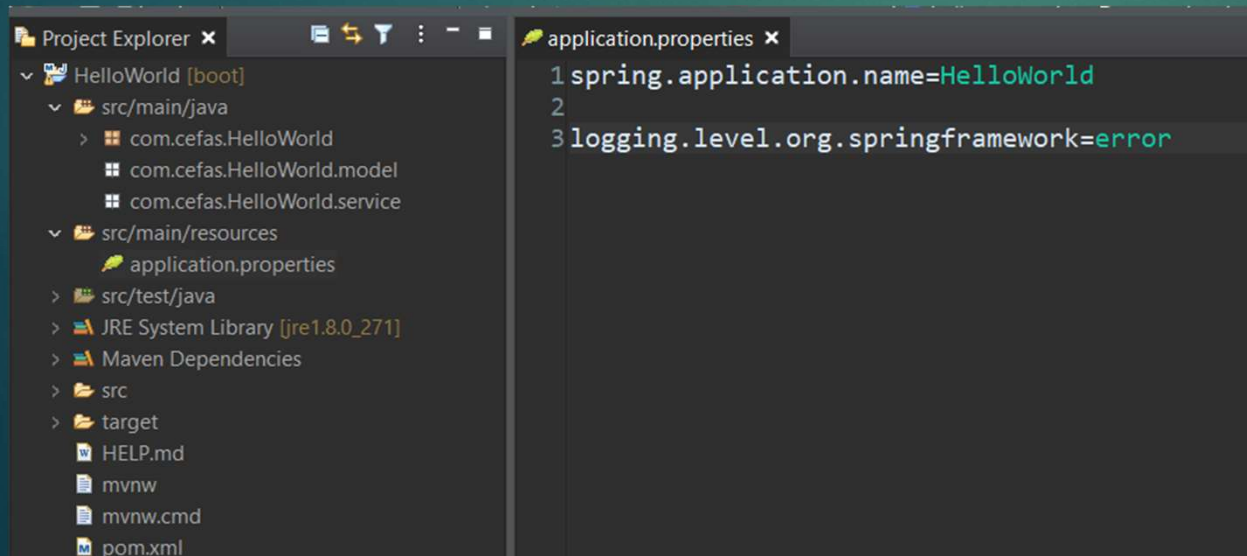
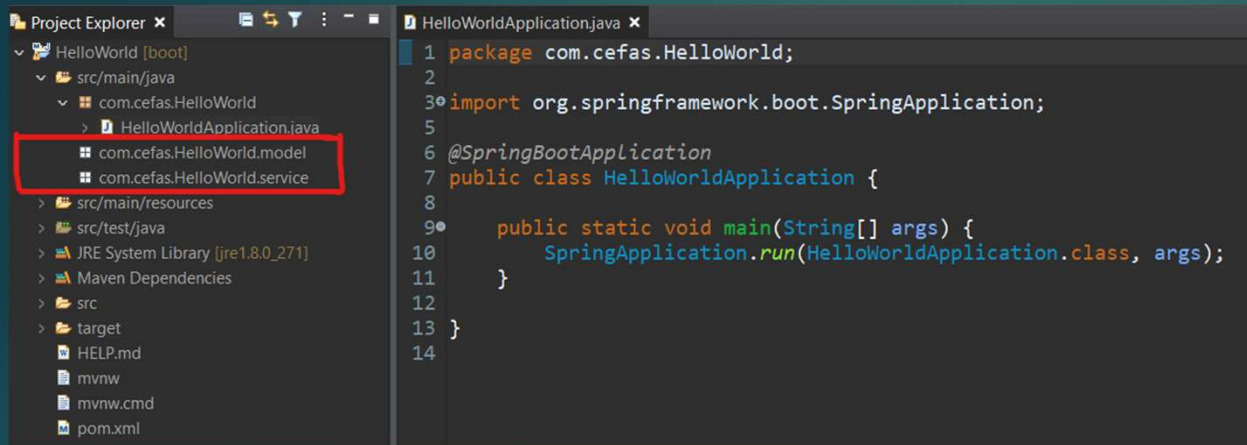
15



- **couche Controller** : gestion des interactions entre l'utilisateur de l'application et l'application ;
- **couche Service** : implémentation des traitements métiers spécifiques à l'application ;
- **couche Repository** : interaction avec les sources de données externes ;
- **couche Model** : implémentation des objets métiers qui seront manipulés par les autres couches.

2- STRUCTURER ET CONFIGURER LE PROJET(2/2)

16



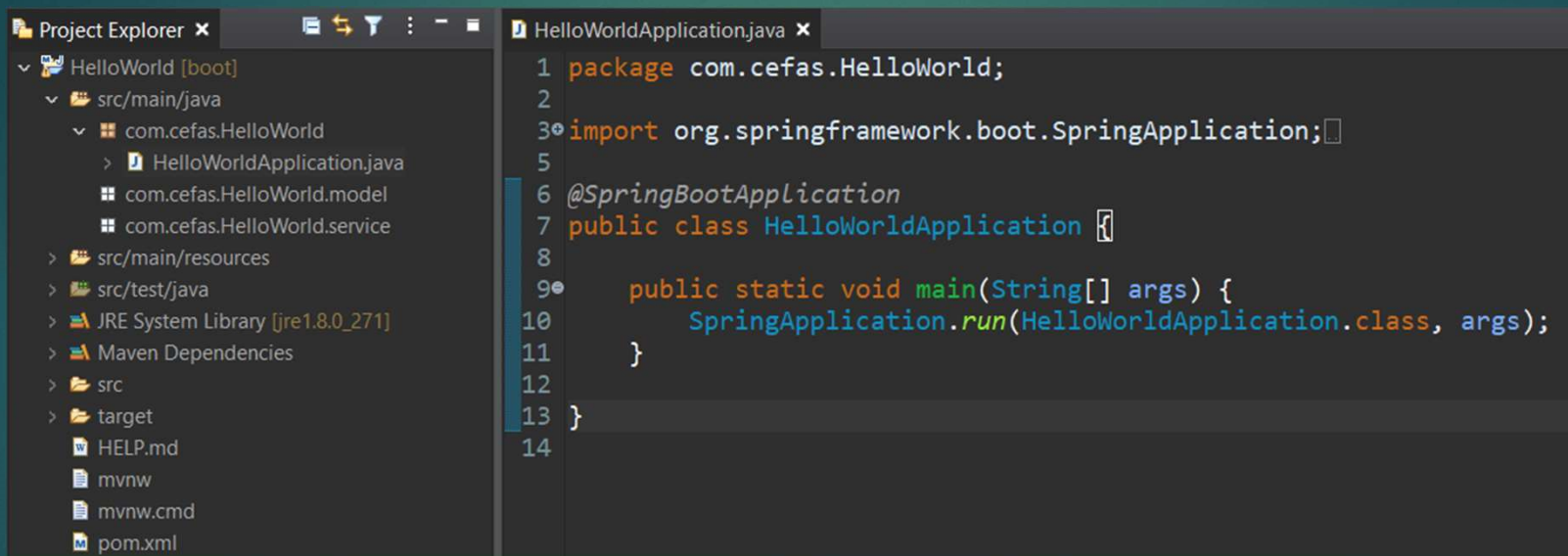
- Configurer le projet en créant deux packages: model et service
- Configurer le fichier application.properties :

1. **spring.application.name** : permet de donner un nom à notre application Spring Boot (c'est informatif).
2. **logging.level.org.springframework=ERROR** : les classes du package org.springframework affichent uniquement les logs de niveau ERROR dans la console (autrement dit, on n'est pas pollué par plein d'informations).

3- ECRIRE VOTRE CODE

17

- L'objectif, le besoin fonctionnel est d'afficher le texte "Hello World !"
- Qu'avons-nous besoin pour atteindre cet objectif ? :C'est savoir écrire dans la console.
- Normalement dans la méthode main:



The screenshot shows an IDE with two panels. The left panel is the Project Explorer, showing a project named 'HelloWorld [boot]' with a directory structure: 'src/main/java' containing 'com.cefes.HelloWorld' (with 'HelloWorldApplication.java', 'HelloWorld.model', and 'HelloWorld.service') and 'src/main/resources'. The right panel shows the 'HelloWorldApplication.java' file with the following code:

```
1 package com.cefes.HelloWorld;
2
3 import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class HelloWorldApplication {
8
9     public static void main(String[] args) {
10         SpringApplication.run(HelloWorldApplication.class, args);
11     }
12 }
13
14
```

- “**SpringApplication.run(HelloWorldApplication.class, args);**”. Cette instruction permet de démarrer notre application, et ce n'est pas une bonne pratique d'ajouter autre chose dans la méthode *main*.

3- ECRIRE VOTRE CODE

18

- L'interface **CommandLineRunner**: Spring Boot fournit cette interface. Implémentation obligatoire de la méthode “**public void run(String... args) throws Exception**”. À partir de là, si la classe est un bean (c'est-à-dire chargée dans le contexte Spring: IoC), Spring Boot exécutera la méthode run à l'exécution du programme.
- Donc 2 méthodes:
 1. soit modifier la classe HelloWorldApplication afin qu'elle implémente CommandLineRunner et la méthode run, avec comme corps de méthode un “System.out.println(“Hello World!”)” ;
 2. soit créer une nouvelle classe qui implémente CommandLineRunner, la méthode run (même corps de méthode), et qui aura une annotation @Component (au-dessus du nom de la classe).

3- ECRIRE VOTRE CODE: Résultat

19

The screenshot displays an IDE with the following components:

- Project Explorer:** Shows the project structure for 'Helloworld [boot]'. The 'src/main/java' directory contains the package 'com.cefes.HelloWorld', which includes 'HelloworldApplication.java', 'Helloworld.model', and 'Helloworld.service'.
- Editor:** Displays the code for 'HelloworldApplication.java'. The code is as follows:

```
1 package com.cefes.HelloWorld;
2
3 import org.springframework.boot.CommandLineRunner;
4
5
6
7 @SpringBootApplication
8 public class HelloworldApplication implements CommandLineRunner{
9
10     public static void main(String[] args) {
11         SpringApplication.run(HelloworldApplication.class, args);
12     }
13
14     @Override
15     public void run(String... args) throws Exception {
16         System.out.println("Hello World !");
17     }
18
19
20 }
21
```
- Outline:** Shows the class hierarchy: 'com.cefes.HelloWorld' containing 'HelloworldApplication', which has methods 'main(String[]): void' and 'run(String...): void'.
- Console:** Shows the output of the application. It starts with a Spring Boot logo and version information (v2.7.14). The output includes:

```
<terminated> Helloworld - HelloworldApplication [Spring Boot App] C:\Program Files\AdoptOpenJDK\jdk-8.0.232.09-hotspot\bin\javaw.exe (14 août 2023 18:54:30 - 18:54:34)

:: Spring Boot :: (v2.7.14)

2023-08-14 18:54:33.512 INFO 1188 --- [main] c.m.t.HelloworldApplication : Starting HelloworldApplication using Java 1.8.0_232 on SNDK6399LAPI
2023-08-14 18:54:33.515 INFO 1188 --- [main] c.m.t.HelloworldApplication : No active profile set, falling back to 1 default profile: "default"
2023-08-14 18:54:34.146 INFO 1188 --- [main] c.m.t.HelloworldApplication : Started HelloworldApplication in 1.163 seconds (JVM running for 2.61)
Hello World !
```

3- ECRIRE VOTRE CODE: Exploiter model et service

20

```
Project Explorer x
Helloworld [boot]
  Spring Elements
  src/main/java
    com.cefes.HelloWorld
      HelloworldApplication.java
      com.cefes.HelloWorld.model
        HelloWorld.java
      com.cefes.HelloWorld.service
src/main/resources
src/test/java
JRE System Library [JavaSE-1.8]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml

HelloWorld.java x
1 package com.cefes.HelloWorld.model;
2
3 public class HelloWorld {
4
5     private String value = "Hello World !";
6
7     public String getValue() {
8
9         return value;
10    }
11
12    @Override
13    public String toString() {
14
15        return value;
16    }
17
18 }
19
```

```
Project Explorer x
Helloworld [boot]
  Spring Elements
  src/main/java
    com.cefes.HelloWorld
      HelloworldApplication.java
      com.cefes.HelloWorld.model
        HelloWorld.java
      com.cefes.HelloWorld.service
        MonService.java
src/main/resources
src/test/java
JRE System Library [JavaSE-1.8]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml

HelloWorld.java x
1 package com.cefes.HelloWorld.model;
2
3 public class HelloWorld {
4
5     private String value = "Hello World !";
6
7     public String getValue() {
8
9         return value;
10    }
11
12    @Override
13    public String toString() {
14
15        return value;
16    }
17
18 }
19

*MonService.java x
1 package com.cefes.HelloWorld.service;
2
3 import org.springframework.stereotype.Component;
4 import com.cefes.HelloWorld.model.HelloWorld;
5
6 @Component
7 public class MonService {
8
9     public HelloWorld getHelloWorld() {
10
11         return new HelloWorld();
12     }
13
14 }
15
```

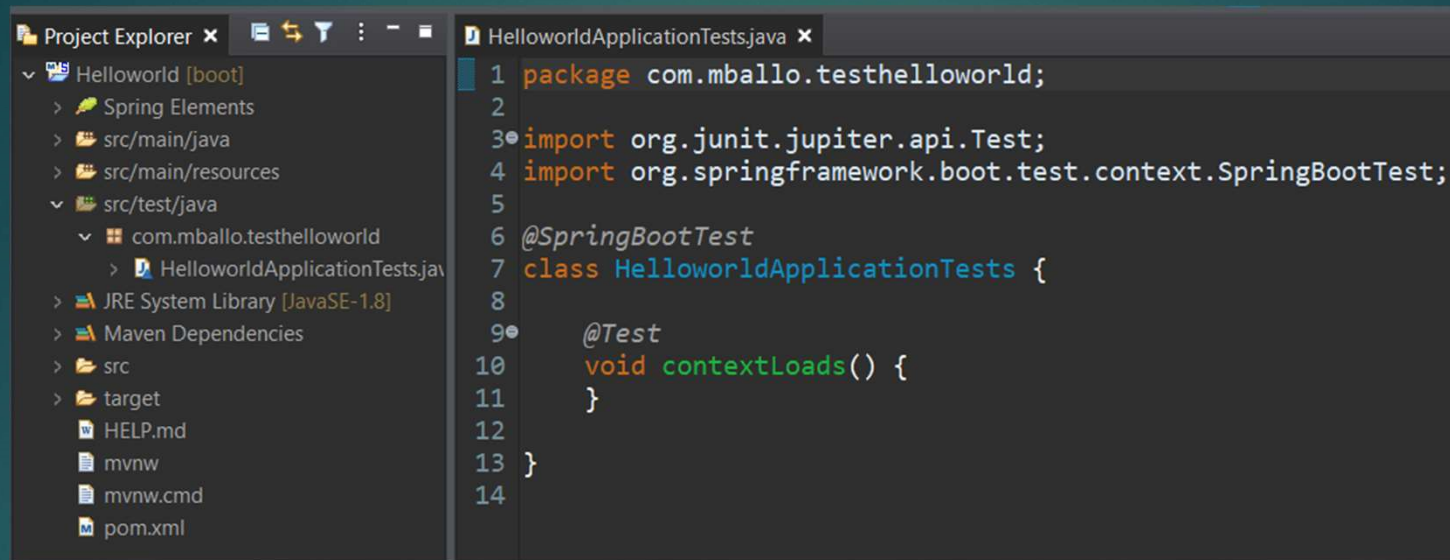
```
Project Explorer x
Helloworld [boot]
  Spring Elements
  src/main/java
    com.cefes.HelloWorld
      HelloworldApplication.java
      com.cefes.HelloWorld.model
        HelloWorld.java
      com.cefes.HelloWorld.service
        MonService.java
src/main/resources
src/test/java
JRE System Library [JavaSE-1.8]
Maven Dependencies
src
target
HELP.md
mvnw
mvnw.cmd
pom.xml

*HelloworldApplication.java x
1 package com.cefes.HelloWorld;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4
5 @SpringBootApplication
6 public class HelloworldApplication implements CommandLineRunner{
7
8     @Autowired
9     private MonService ms;
10
11     public static void main(String[] args) {
12         SpringApplication.run(HelloworldApplication.class, args);
13     }
14
15     @Override
16     public void run(String... args) throws Exception {
17
18         HelloWorld hw = ms.getHelloWorld();
19         System.out.println(hw);
20     }
21
22 }
23
24
25
26
27
28
29
```

4-TESTER ET DEPLOYER

1-TESTER

21



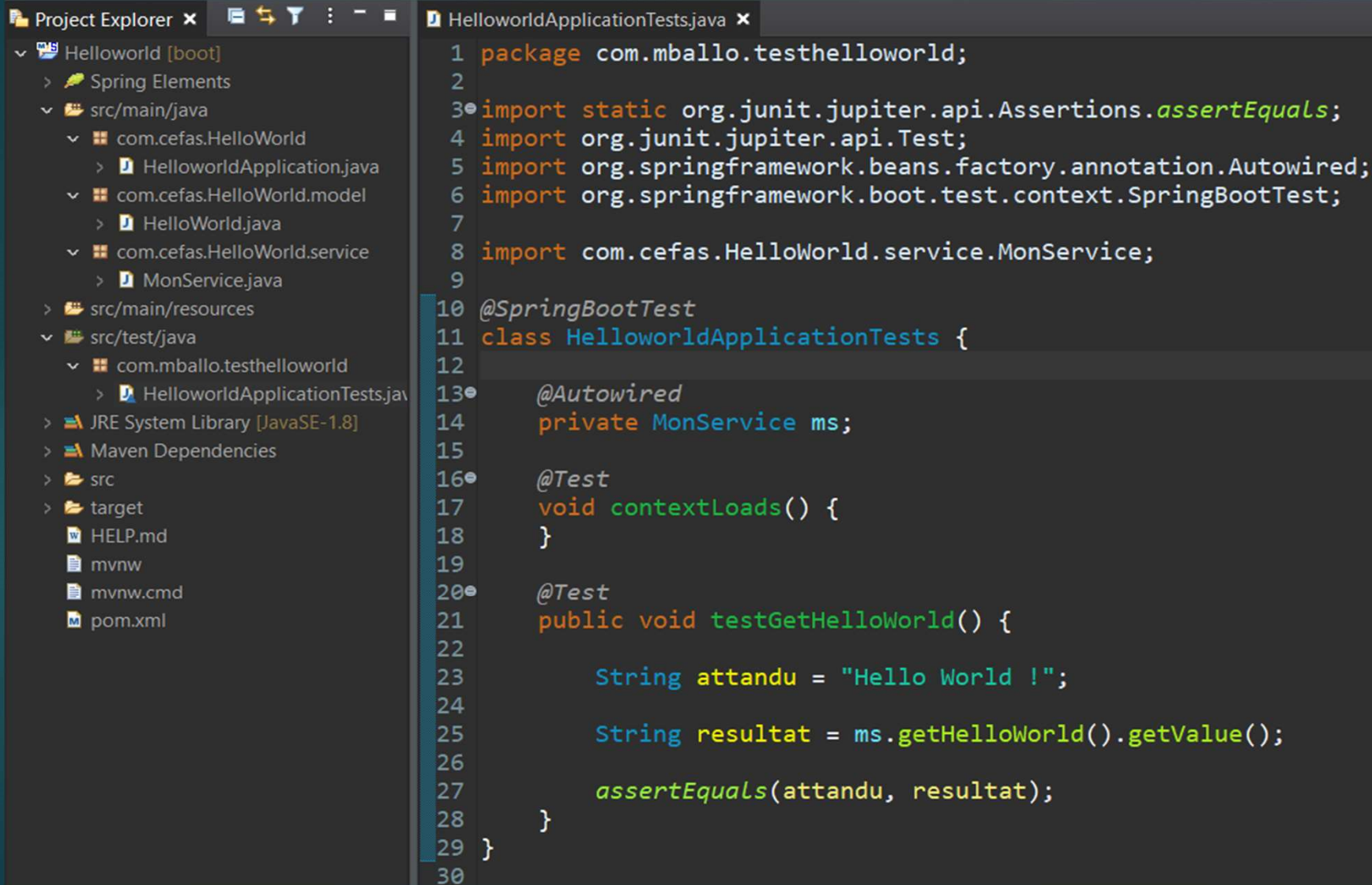
The screenshot shows an IDE with two panels. The left panel, 'Project Explorer', displays a project named 'Helloworld [boot]'. It contains a 'src/test/java' directory with a package 'com.mballo.testhelloworld' and a class 'HelloworldApplicationTests.java'. The right panel shows the code of 'HelloworldApplicationTests.java'.

```
1 package com.mballo.testhelloworld;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class HelloworldApplicationTests {
8
9     @Test
10    void contextLoads() {
11    }
12
13 }
14
```

- **@SpringBootTest** : Elle permet lors de l'exécution des tests d'initialiser le contexte Spring. Les beans de notre application peuvent alors être utilisés.
- La méthode **contextLoads()** annoté avec **@Test** provient de Junit: son unique objectif est de **vérifier que le contexte Spring se charge bien**. Sans cette méthode la classe de test ne peut être exécutée. Donc lors de l'exécution de cette méthode, le contexte Spring sera chargé, et si ce dernier rencontre une erreur, alors l'exécution de la classe de test retournera une erreur.

1-TESTER

22



The screenshot shows an IDE with two panes. The left pane, titled 'Project Explorer', displays the project structure for 'Helloworld [boot]'. It includes a 'src/main/java' directory with packages 'com.cefas.HelloWorld' (containing 'HelloWorldApplication.java', 'HelloWorld.model', 'HelloWorld.java', and 'MonService.java') and 'src/main/resources'. There is also a 'src/test/java' directory with a package 'com.mballo.testhelloworld' containing 'HelloworldApplicationTests.java'. Other files like 'HELP.md', 'mvnw', 'mvnw.cmd', and 'pom.xml' are listed at the bottom. The right pane shows the code for 'HelloworldApplicationTests.java'. The code is as follows:

```
1 package com.mballo.testhelloworld;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import org.junit.jupiter.api.Test;
5 import org.springframework.beans.factory.annotation.Autowired;
6 import org.springframework.boot.test.context.SpringBootTest;
7
8 import com.cefas.HelloWorld.service.MonService;
9
10 @SpringBootTest
11 class HelloworldApplicationTests {
12
13     @Autowired
14     private MonService ms;
15
16     @Test
17     void contextLoads() {
18     }
19
20     @Test
21     public void testGetHelloWorld() {
22
23         String attandu = "Hello World !";
24
25         String resultat = ms.getHelloWorld().getValue();
26
27         assertEquals(attandu, resultat);
28     }
29 }
30
```

2-DEPLOIEMENT

23

Le déploiement correspond aux étapes qui permettent de passer de l'environnement de développement à l'environnement d'exécution(environnement de test ou de production).

3 methods existent:

1. A travers l'IDE avec "Run As" → Spring Boot App
2. Avec `mvn spring-boot:run`
3. En exécutant le JAR grace à la commande `java -jar`

NB: En milieu professionnel on livrera généralement un JAR, et c'est ce dernier qui sera exécuté. Les méthodes de déploiement et d'exécution via l'IDE et Maven sont surtout utiles pour les développeurs.